

IMU SENSÖR SİMÜLASYONU

Teknik Vaka Çalışması Raporu

Bu rapor, IMU verilerini yazılımsal olarak üreten, paketleyen, filtreleyen ve hata senaryoları altında test eden C tabanlı uygulamanın tasarımını ve sonuçlarını özetlemektedir.

Programlama Dili	C
Simülasyon Süresi	5000 ms (5 saniye)
Örnekleme Periyodu	10 ms (100 Hz)
Hedef Ortam	Linux / Windows (donanımsız)

1. Proje amacı ve kapsamı

Bu çalışma, IMU (Inertial Measurement Unit) verilerini gerçek donanım olmadan yazılımsal olarak üretmek, bu verileri sensörden geliyormuş gibi paketlemek, filtrelemek ve farklı haberleşme sorunları altında test etmek amacıyla hazırlanmıştır. Uygulama C dili ile modüler bir yapıda geliştirilmiştir.

Proje boyunca özellikle şu başlıklara odaklanılmıştır: veri üretimi, veri yapısı seçimi, yazılımsal protokol tasarımı, filtreleme, periyodik işleme mantığı, işlem süresi ölçümü ve hata senaryolarının gözlemlenmesi.

2. Seçilen IMU sensörü ve seçim gerekçesi

Referans sensör olarak MPU-6050 seçilmiştir. Bu sensör, 3 eksenli ivmeölçer ve 3 eksenli jiroskopu aynı pakette birleştirdiği için eğitim ve simülasyon çalışmalarında sık kullanılan bir örnektir. Geniş dokümantasyon desteğine sahip olması, veri formatının iyi bilinmesi ve temel IMU kavramlarını göstermek için yeterli olması nedeniyle tercih edilmiştir.

2.1 İvmeölçer çalışma prensibi

İvmeölçer, MEMS yapısı içindeki hareketli kütlenin yer değiştirmesine bağlı kapasitans değişimini ölçer. Bu sayede doğrusal ivme ve sabit durumda yerçekimi bileşenleri elde edilir. Bu projede sensörün sabit durumda yerçekimi vektörünü ölçtüğü kabul edilmiştir; dolayısıyla roll ve pitch açıları ivmeölçer verisinden tahmin edilebilmektedir.

2.2 Jiroskop çalışma prensibi

Jiroskop, Coriolis etkisini kullanarak açısal hızı ölçer. Kısa vadede hızlı ve kararlı bilgi üretir; ancak bias ve drift nedeniyle tek başına kullanıldığında zamanla açı hatası birikir. Bu nedenle jiroskop verisi tamamlayıcı filtre içinde ivmeölçer ile birleştirilmiştir.

3. Matematiksel model

3.1 İdeal hareket modeli

Simülasyonda 0.5 Hz frekansta sinüzoidal bir hareket kullanılmıştır. Roll, pitch ve yaw değerleri derece cinsinden aşağıdaki gibidir:

```
roll(t) = 20 * sin(2*pi*0.5*t)
pitch(t) = 15 * sin(2*pi*0.5*t + pi/6)
yaw(t) = 8 * sin(2*pi*0.5*t + pi/4)
```

Pitch ve yaw eksenlerindeki faz kaymaları, eksenlerin birbirinden bağımsız görünmesini sağlar ve filtre çıktılarının karşılaştırılmasını daha anlamlı hale getirir.

3.2 İvmeölçer modeli

İdeal durumda yalnızca yerçekimi etkisi kabul edilmiştir. Roll ve pitch açılarına göre ivme bileşenleri aşağıdaki gibi hesaplanır:

```
ax = -g * sin(pitch)
ay = g * sin(roll) * cos(pitch)
az = g * cos(roll) * cos(pitch)
```

3.3 Jiroskop modeli

Jiroskop verisi, ideal açılarının türevi alınarak rad/s cinsinden üretilmiştir. Kod içinde önce ideal açısal hız hesaplanmakta, ardından buna gürültü, bias ve drift eklenmektedir.

3.4 Sensör hata modeli

Gerçek sensör davranışını taklit etmek için ivmeölçer tarafına Gauss gürültüsü; jiroskop tarafına ise Gauss gürültüsü, sabit bias ve eksen bazlı drift eklenmiştir. Drift yalnızca X ekseninde değil, üç ekseninde tanımlıdır.

Hata türü	Kodda kullanılan parametre	Açıklama
İvmeölçer gürültüsü	$\sigma = 0.20$	Her eksene bağımsız normal dağılımlı gürültü eklenir.
Jiroskop gürültüsü	$\sigma = 0.10$ dps	Her örnekte açısal hızı küçük rastgele sapma eklenir.
Sabit bias	$X=+0.20, Y=-0.15, Z=+0.10$ dps	Eksen bazlı ofset hatasıdır.
Drift	$X=+0.020, Y=-0.015, Z=+0.010$ dps/s	Zamanla biriken sapmayı temsil eder.

3.5 Ham veri dönüşümü

Fiziksel birimlerden ham sensör verisine dönüşüm yapılırken MPU-6050 ölçek katsayıları esas alınmıştır. İvmeölçer için 16384 LSB/g, jiroskop için 131 LSB/dps kullanılmıştır. Jiroskop dönüşümünde sensörün ± 250 deg/s ölçüm aralığı kullanılmıştır.

Jiroskop verisi üretilirken önce rad/s cinsinden hesaplanan açısal hız derece/saniyeye çevrilmiş, ardından ölçeklenerek ham veri elde edilmiştir:

```
gx_dps = RAD2DEG(gx_rad_s)
```

```
raw_gx = gx_dps * 131
```

Ham verilerden fiziksel birimlere dönüşüm ise aşağıdaki şekilde yapılmaktadır:

```
acc = raw_acc / 16384 * 9.80665
```

```
gyro = raw_gyro / 131
```

4. Yazılım mimarisi

Kod tabanı modüler hazırlanmıştır. Her dosya belirli bir sorumluluk taşır ve ana akış senaryo katmanında bir araya gelir.

Dosya	Sorumluluk	Öne çıkan işlevler
imu_types.h	Temel sabitler ve veri tipleri	Pose, ImuData, RawData, Packet, Stats
imu_sim.c/.h	İdeal veri ve hata ekleme	ideal_pose, make_ideal_sample, add_sensor_errors
filters.c/.h	Filtreleme ve açı hesabı	avg_apply, lpf_apply, update_complementary
packet.c/.h	Paket üretme ve ayrıştırma	make_packet, parse_packet
scenario.c/.h	Senaryo akışı ve çıktı yönetimi	inject_packet, run_scenario, print_stats
imu_utils.c/.h	Yardımcı işlevler ve queue	now_us, rand_gauss, queue_push, queue_pop

4.1 Veri akışı

- İdeal IMU örneği üretilir.
- Örnek üzerine gürültü, bias ve drift eklenir.
- Veri ham formata çevrilir ve paketlenir.
- Paket ilgili senaryoya göre kuyruğa verilir, düşürülür, bozulur veya geciktirilir.
- Kuyruktan alınan paket doğrulanır ve fiziksel değerlere çevrilir.
- Filtreler uygulanır, ardından roll ve pitch açıları hesaplanır.
- İşlem süresi ölçülür ve belirli periyotlarda terminal çıktısı verilir.

4.2 Queue yapısı

Paket aktarımı için sabit boyutlu bir FIFO dizi kullanılmıştır. Yapı dinamik bellek kullanmadığı için gömülü sistem yaklaşımına uygundur.

5. Paket yapısı ve yazılımsal protokol

Sensörden geliyormuş gibi davranan veri, 24 baytlık sabit uzunluklu paketlere yerleştirilmiştir. Bayt sıralaması little-endian olacak şekilde düzenlenmiştir.

Ofset	Boyut	Alan	Açıklama
0-1	2	0xAA 0x55	Senkronizasyon baytları
2-3	2	SEQ	Paket sıra numarası
4-7	4	TIME_MS	Zaman damgası
8-19	12	AX..GZ	6 adet int16 ham IMU verisi
20-22	3	RESERVED	İleri kullanım için ayrılmış alan
23	1	CHECKSUM	İlk 23 baytın XOR değeri

Paket doğrulama sırasında önce senkronizasyon baytları, ardından checksum kontrol edilir. Checksum tutmazsa paket işlenmeden reddedilir.

Bozuk paket senaryosu düzeltilmesi: Bozuk paket senaryosunda paket oluşturulduktan sonra 10. bayt XOR ile değiştirilir. Bu nedenle paket içeriği ile son bayttaki checksum artık uyuşmaz ve parse_packet işlevi paketi checksum hatası olarak reddeder.

6. Filtreleme yaklaşımı

6.1 Moving Average

İvmeölçer verisi kısa süreli dalgalanmalara daha açık olduğu için eksen bazlı moving average filtresi kullanılmıştır. Pencere boyutu 5 seçilmiştir. Bu seçim hem kodu sade tutmakta hem de yüksek frekanslı gürültüyü bastırmaktadır.

6.2 Low-Pass Filter

Jiroskop tarafında birinci dereceden IIR low-pass filtre kullanılmıştır. LPF_ALPHA değeri 0.35 seçilmiştir. Böylece ani sıçramalar yumuşatılırken jiroskobun kısa vadeli dinamik avantajı korunmuştur.

6.3 Complementary Filter

Roll ve pitch açıları için tamamlayıcı filtre uygulanmıştır. Jiroskop kısa vadeli değişimleri taşıırken, ivmeölçer uzun vadeli referansı sağlar.

```
roll_comp = a * (roll_comp + gx * dt) + (1-a) * roll_acc  
pitch_comp = a * (pitch_comp + gy * dt) + (1-a) * pitch_acc  
a = 0.98
```

Yaw açısı tamamlayıcı filtre ile düzeltilmemiştir. Bunun nedeni, yalnızca ivmeölçer ve jiroskop kullanıldığında yaw için mutlak referans bulunmamasıdır. Kodda yaw yalnızca jiroskop entegrasyonu ile izleme amaçlı tutulmaktadır.

7. Zamanlama ve işlem süresi ölçümü

Ana döngü 10 ms adımlarla ilerlemektedir. Her adımda üretim, paketleme, senaryo enjeksiyonu, kuyruktan alma, doğrulama, filtreleme ve çıktı hazırlama işlemleri gerçekleştirilir. Bu yaklaşım bare-metal zamanlayıcı mantığına benzer ve donanım gerektirmeden simülasyon yapılmasına olanak verir.

İşlem süresi mikrosaniye cinsinden ölçülmektedir. Windows tarafında QueryPerformanceCounter kullanılmaktadır.

8. Test senaryoları ve gözlenen sonuçlar

Program dört farklı senaryo ile çalıştırılmış ve terminal çıktısından elde edilen özet değerler aşağıda sunulmuştur. Ölçümlerde özellikle paket sayıları, checksum hataları ve roll/pitch RMSE değerleri takip edilmiştir.

Senaryo	İşlenen	Kayıp	Bozuk	Gecikmeli	Checksum	Ort. işlem (us)	Roll RMSE	Pitch RMSE
Normal	501	0	0	0	0	0.183	0.694	0.700
Paket kaybı	496	5	0	0	0	0.151	0.761	0.611
Bozuk paket	496	0	5	0	5	0.157	0.877	0.657
Gecikmeli paket	500	0	0	5	0	0.162	1.127	0.878

8.1 Senaryoların yorumu

- Normal senaryoda sistem tüm paketleri işlemiş ve düşük RMSE değerleri elde edilmiştir. Bu sonuç, filtre seçiminin temel işlevi yerine getirdiğini göstermektedir.
- Paket kaybı senaryosunda her saniye bir paket düşürülmüş, buna rağmen filtre çıkışı büyük ölçüde kararlı kalmıştır.
- Bozuk paket senaryosunda 5 paket checksum hatası ile elenmiştir. Bu, paket doğrulama mekanizmasının çalıştığını göstermektedir.
- Gecikmeli paket senaryosunda geciken paket kuyruğa sonradan bırakıldığı için kısa süreli bir birikme oluşmuştur. Mevcut akışta her çevrimde en fazla bir paket işlendiğinden simülasyon sonunda kuyrukta bir paket kalmış ve işlenen paket sayısı 500 olmuştur.

9. Çıktı Analizi ve Değerlendirme

Simülasyon çıktısı incelendiğinde, sistemin genel olarak doğru ve tutarlı çalıştığı görülmektedir. Her zaman adımında ideal açı değerleri, filtrelenmiş açı değerleri, sensör verileri ve işlem süresi birlikte sunulmaktadır.

Başlangıçta (0 ms) ideal değerler ile filtre çıktısı arasında fark bulunmaktadır. Bu durum, filtrenin henüz kararlı hale gelmemesinden kaynaklanmaktadır. Ancak ilerleyen zaman adımlarında complementary filter çıktılarının ideal değerlere yaklaştığı açıkça görülmektedir.

Jiroskop entegrasyonu ile elde edilen açılar zamanla hataya kayma eğilimindedir. Bunun nedeni simülasyonda eklenen drift ve bias etkileridir. Complementary filter ise bu hatayı azaltarak daha stabil sonuçlar üretmektedir.

İvmeölçer verisi daha gürültülü olmasına rağmen uzun vadede doğru referans sağlar. Jiroskop verisi ise kısa vadede daha düzgün sonuç verir ancak zamanla sapar. Filtre, bu iki veriyi birleştirerek daha dengeli bir çıktı üretmektedir.

Ham veriler (Raw Acc, Raw Gyro) ile fiziksel birimler (Physical Acc, Physical Gyro) karşılaştırıldığında dönüşümlerin doğru yapıldığı görülmektedir.

Ham verilerden fiziksel birimlere dönüşüm aşağıdaki şekilde yapılmıştır:

$\text{Physical_acc} = \text{raw_acc} / 16384 * 9.80665$

$\text{Physical_gyro} = \text{raw_gyro} / 131$

Çıktıda yer alan Raw değerler, sensörün ürettiği ham sayısal verilerdir (LSB cinsinden). Physical değerler ise bu ham verilerin sensör ölçek katsayıları kullanılarak gerçek fiziksel birimlere dönüştürülmüş halidir. İvmeölçer verileri m/s^2 , jiroskop verileri ise deg/s cinsinden ifade edilmektedir.

Bu dönüşüm sayesinde veriler filtreleme ve açı hesaplama işlemlerinde kullanılabilir hale gelmektedir. Ayrıca filtrelenmiş veriler, ham verilere göre daha yumuşak ve kararlıdır.

Sonuç olarak, sistemin veri üretimi, filtreleme ve işleme adımlarının birlikte doğru çalıştığı ve filtreleme sayesinde daha kararlı açı tahminleri elde edildiği görülmektedir.

10. Sonuç

Bu vaka çalışması, donanım olmadan IMU veri zincirinin önemli parçalarını gerçekçi biçimde modelleyen bir C uygulaması ortaya koymaktadır. Veri üretimi, paketleme, filtreleme, hata senaryosu testleri ve işlem süresi ölçümü gereksinimleri ana hatlarıyla karşılanmıştır.

Teknik değerlendirme açısından çalışma; sensör verisi modelleme, temel filtreleme bilgisi, statik veri yapıları kullanımı ve hata senaryolarını düşünme becerisi gösterdiği için güçlü bir temel sunmaktadır. En kritik geliştirme alanı, gecikmeli senaryoda kuyruk boşaltma akışının iyileştirilmesi ve kuyruk yapısının halka tampon haline getirilmesidir.